

Лабораторная работа №6 – «Конфигурация и установка ядра Linux»

Цель: измерение зависимости времени сборки ядра Linux от количества используемых потоков и получение работающего сконфигурированного ядра

Задачи:

- 1) Подготовка системы и установка необходимых пакетов
- 2) Установка исходного кода ядра и распаковка архива
- 3) Конфигурация ядра
- 4) Сборка ядра
- 5) Установка ядра
- 6) Проверка правильности установки
- 7) Разработка сценария для измерения времени сборки

Аппаратная платформа: AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx

Программная платформа: Ubuntu 24.04.3 LTS x86_64

Подготовка системы и установка необходимых пакетов

```
sudo apt update  
sudo apt install build-essential libncurses-dev bison flex libssl-dev  
libelf-dev bc
```

- *build-essential* – набор компиляторов и инструментов C/C++ (gcc, g++, make, dpkg-dev, libc6-dev, ld, as, ar, nm);
- *libncurses-dev* – библиотека для реализации TUI;
- *bison + flex* – инструменты для работы с лексическими представлениями программ на C;
- *libssl-dev* – зависимость для работы с шифрованием SSL;
- *libelf-dev* – библиотека для чтения и записи файлов ELF;
- *bc* – интерактивный калькулятор;

Установка исходного кода ядра

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-  
6.18.10.tar.xz  
tar -xvf linux-6.18.10.tar.xz  
cd linux-6.18.10
```

tar – утилита для создания и распаковки архивов.

- `-x` – извлечение файлов из архива;
- `-v` – вывод списка обрабатываемых файлов в терминал;
- `-f` – указание на то, что далее следует имя файла архива;

Конфигурация ядра

```
cp /boot/config-$(uname -r) .config
make menuconfig
```

- *defconfig* – стандартная конфигурация для текущей архитектуры;
- *tinyconfig* – минимально возможное ядро (только самое необходимое для загрузки);
- *olddefconfig* – текущая конфигурация + значения по умолчанию для новых опций;

Сборка ядра

```
make -j$(nproc)
```

nproc – количество ядер в системе;

Установка ядра

```
sudo make install
sudo update-grub // обновление конфигурации загрузчика GRUB
```

Проверка правильности установки

```
reboot
uname -r // после перезагрузки выбираем новое ядро и проверяем
версию
```

Разработка сценария для измерения времени сборки

```
#!/bin/bash

N=$(nproc)

RESULTS_FILE="results.txt"
echo "Потоки (-j) | Время сборки (сек)" > $RESULTS_FILE
cd ~/linux-6.18.10

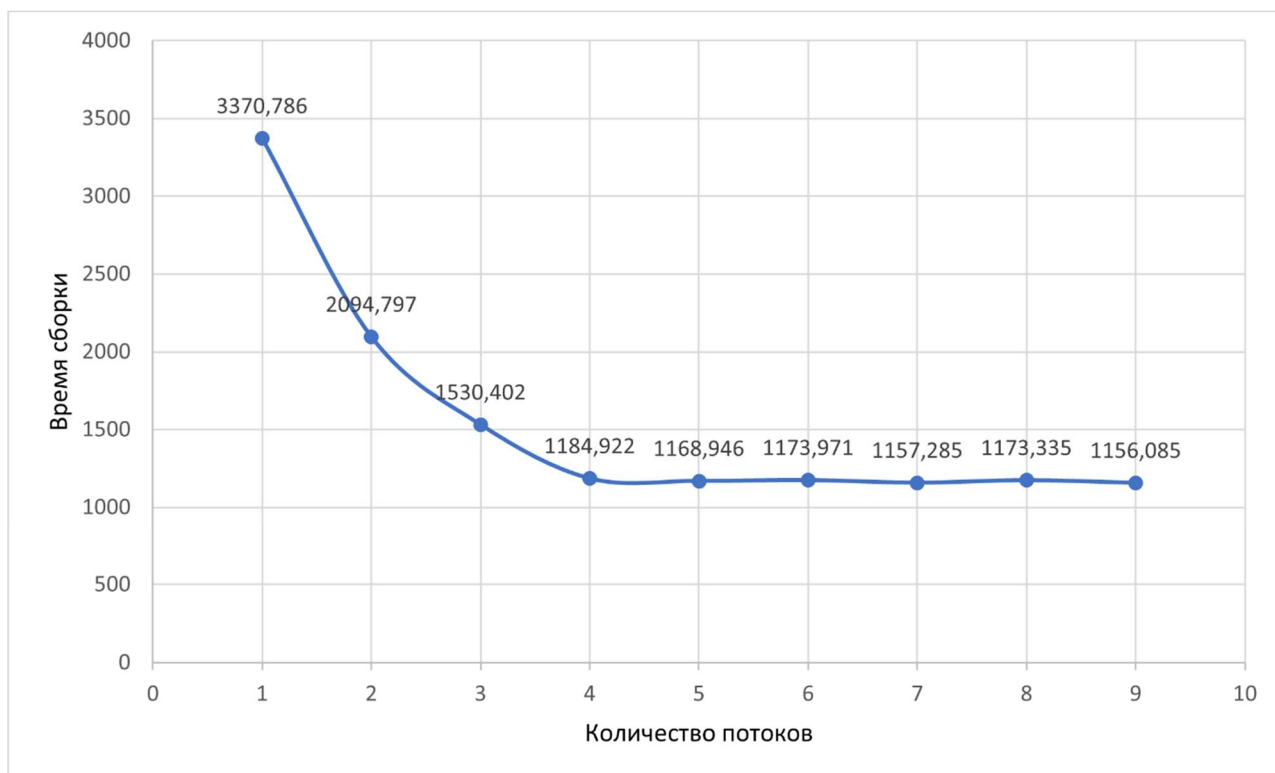
for (( i=0; i <= 2*N+1; i++ ))
do
    echo "Запуск сборки с -j$i ..."
    make clean > /dev/null 2>&1
    TIMEFORMAT=%R
    BUILD_TIME=$( { time make -j$i > /dev/null 2>&1; } 2>&1 )
```

```
echo "Сборка с -j$i заняла $BUILD_TIME секунд"
echo "-j$i | $BUILD_TIME" >> ~/LAB6/$RESULTS_FILE
done

echo "Тестирование завершено. Результаты находятся в файле
$RESULTS_FILE"
```

Полученные результаты и график

Потоки (-j)	Время сборки (сек)
-j0	0,029
-j1	3370,786
-j2	2094,797
-j3	1530,402
-j4	1184,922
-j5	1168,946
-j6	1173,971
-j7	1157,285
-j8	1173,335
-j9	1156,085



Вывод: лучшее время – **1156,085 сек (-j9)**; начиная с -j4 время сборки практически не меняется